вМинистерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский
политехнический университет»**

Электротехнический факультет
Кафедра «Информационные технологии и автоматизированные системы»
направление подготовки: 09.03.01 – «Информатика, вычислительная
техника»

УТВЕРЖДАЮ
Зав. кафедрой ИТАС

докт. экон. наук, профессор

_____(Р.А.Файзрахманов)

«___» _____ 2026 г.

Дискретная математика
Лабораторная работа №2

Выполнил студент гр. ИВТ-25-1б

Тагиров Вадим Эльвирович
(Фамилия, Имя, Отчество)

_____
(подпись)

Проверил:

Ст. Преп. Каф. ИТАС
Рустамханова. Г. И.
(должность, Ф.И.О руководителя от кафедры)

_____ _____
(оценка) (подпись)

_____
(дата)

г. Пермь 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....3
1 Теоретическая часть.....4
1.1 Основные понятия теории отношений.....4
1.2 Свойства бинарных отношений.....4
1.3 Типы отношений.....4
2 Практическая часть.....5
2.1 Описание решение.....5
2.2 Алгоритмы проверки свойств.....5
2.3 Чтение данных из файлов.....6
2.4 Формирование отчета.....6
3 Результаты тестирования.....7
ЗАКЛЮЧЕНИЕ.....12
Приложение.....13
ПРИЛОЖЕНИЕ А.....14

ВВЕДЕНИЕ

Лабораторная работа посвящена разработке программного обеспечения для определения свойств бинарных отношений и их классификации на основе матриц смежности.

Входные данные представляют собой квадратные матрицы размерности 6×6, состоящие из нулей и единиц. Единица на пересечении строки и столбца означает наличие отношения между соответствующими элементами, ноль — его отсутствие.

Работа выполнена на языке программирования Python. Программа позволяет считывать матрицы из текстовых файлов, автоматически проверять наличие шести основных свойств отношений и определять тип отношения (эквивалентность, порядок и его виды).

1 Теоретическая часть

1.1 Основные понятия теории отношений

Бинарное отношение на множестве задается как набор упорядоченных пар элементов. В данной работе отношение удобно представлять в виде матрицы смежности. Элементы главной диагонали такой матрицы показывают наличие связи элемента с самим собой.

1.2 Свойства бинарных отношений

1. Рефлексивность — каждый элемент связан сам с собой (вся главная диагональ состоит из единиц).

2. Антирефлексивность — ни один элемент не связан сам с собой (вся главная диагональ состоит из нулей).

3. Симметричность — если элемент A связан с B, то и B связан с A (матрица симметрична относительно главной диагонали).

4. Антисимметричность — если A связан с B и B связан с A, то $A = B$ (вне диагонали не может быть двух единиц на симметричных позициях).

5. Транзитивность — если A связан с B, а B связан с C, то A связан с C (если есть единицы на путях $i \to j$ и $j \to k$, то должна быть единица на $i \to k$).

6. Связность (полнота) — для любой пары элементов A и B либо A связан с B, либо B связан с A (нет пар с нулями в обе стороны).

1.3 Типы отношений

1) Эквивалентность = рефлексивность + симметричность + транзитивность.

2) Порядок = антисимметричность + транзитивность.

3) Строгий порядок = антирефлексивность + порядок.

4) Полный (линейный) порядок = связность + порядок.

5) Частичный порядок = порядок (без связности).

2 Практическая часть

2.1 Описание решение

Программа построена на двух классах. Класс *Matrix* — описывает одну матрицу.

- Поля: name (имя матрицы), data (двумерный список), n (размерность).
- Методы: is_reflexive, is_antireflexive, is_symmetric, is_antisymmetric, is_transitive, is_connected — реализуют проверки свойств.
- Метод get_type — на основе проверок возвращает строку с типом отношения.
- Метод __str__ — форматирует матрицу для красивого вывода на экран.

Класс *App* — отвечает за интерфейс и хранение.

- Поля: matrixes (словарь для хранения матриц).
- Методы: load_from_file (чтение файла), add_matrix (загрузка), print_report (вывод свойств), analyze_one и analyze_all (анализ), manager (главное меню).

2.2 Алгоритмы проверки свойств

Рефлексивность/Антирефлексивность: цикл по индексам i от 0 до n-1. Проверяются только элементы диагонали data[i][i].

- Симметричность/Антисимметричность: двойной цикл по i и j. Сравниваются элементы data[i][j] и data[j][i].
- Транзитивность: тройной цикл по i, j, k. Ищутся совпадения data[i][j] == 1 и data[j][k] == 1, после чего проверяется data[i][k] == 1.
- Связность: двойной цикл (где i != j). Ищутся случаи, когда data[i][j] == 0 и data[j][i] == 0.

Для реализации проверок в Python используются встроенная функция all() и генераторы списков, что делает код лаконичным и быстрым.

2.3 Чтение данных из файлов

Метод load_from_file принимает имя файла, открывает его, построчно считывает данные. Каждая строка разбивается методом .split(), элементы преобразуются в целые числа и сохраняются в виде вложенного списка. Если файл не найден, программа выдает сообщение об ошибке.

2.4 Формирование отчета

При анализе программа выводит:

1) Саму матрицу.

2) Результат проверки каждого из шести свойств (Да/Нет).

3) Итоговый тип отношения.

3 Результаты тестирования

Для проверки работоспособности программы было проведено тестирование на предоставленных файлах с матрицами. Внешний вид главного меню программы представлен на рисунке 1.

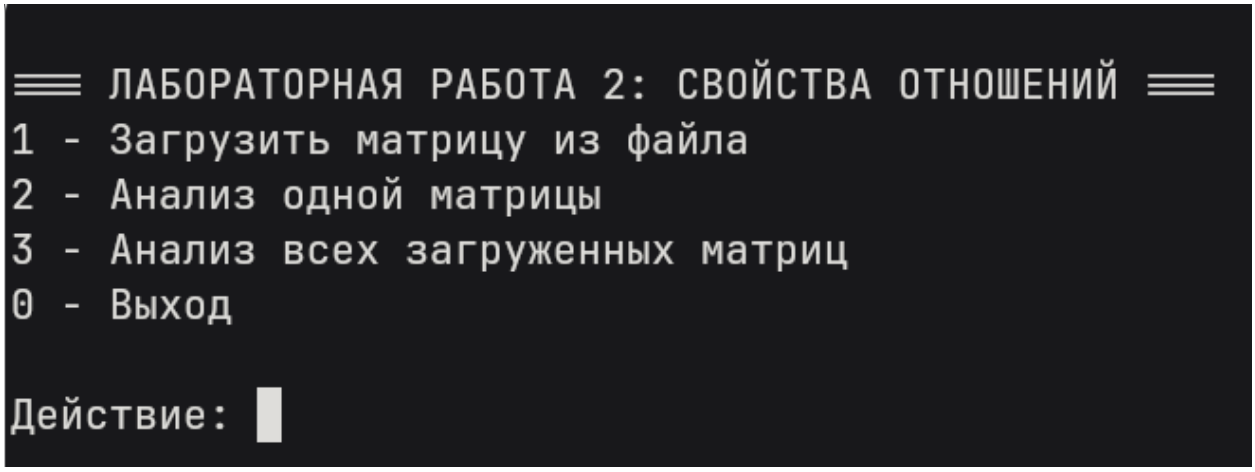


Рисунок 1 — Главное меню программы

Для начала работы необходимо загрузить матрицу из файла. Процесс загрузки матрицы m1 через соответствующее меню показан на рисунке 2.

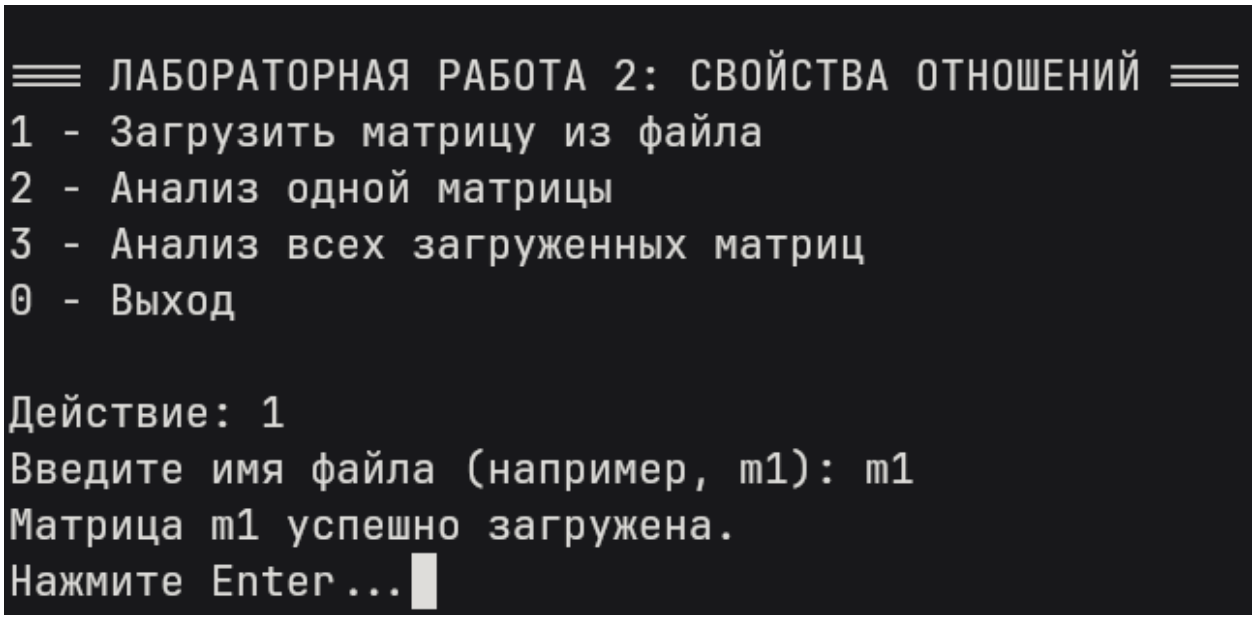


Рисунок 2 — Процесс загрузки матрицы из файла

После загрузки программа позволяет провести анализ свойств выбранной матрицы. На рисунке 3 показан результат анализа матрицы m1,

которая обладает свойствами рефлексивности, симметричности и транзитивности, что соответствует типу «Частичный порядок».

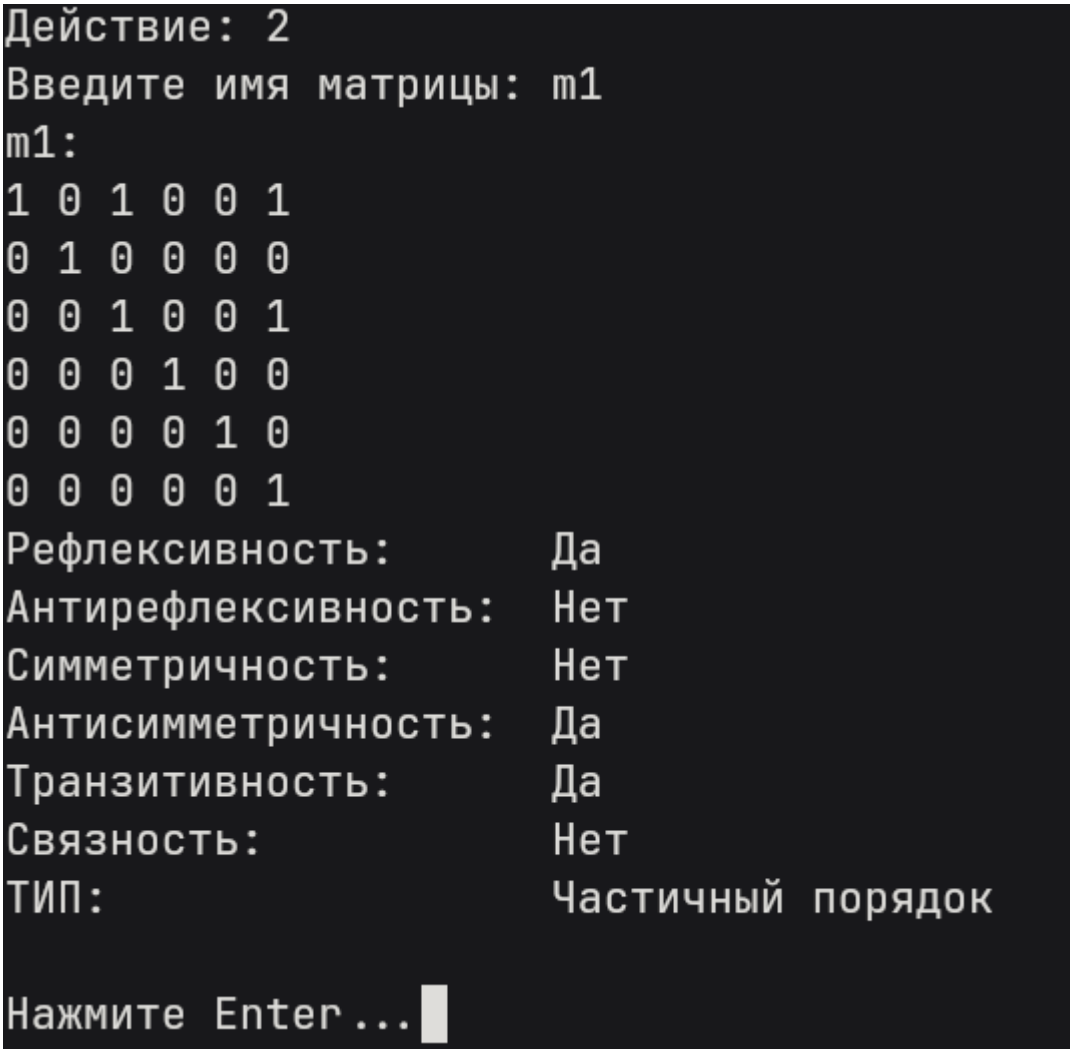


Рисунок 3 — Результат анализа матрицы m1

Далее была проверена матрица m2. Результат её анализа (рисунок 4) показывает наличие антирефлексивности, антисимметричности и транзитивности, на основании чего программа верно определила тип отношения как «Эквивалентность».

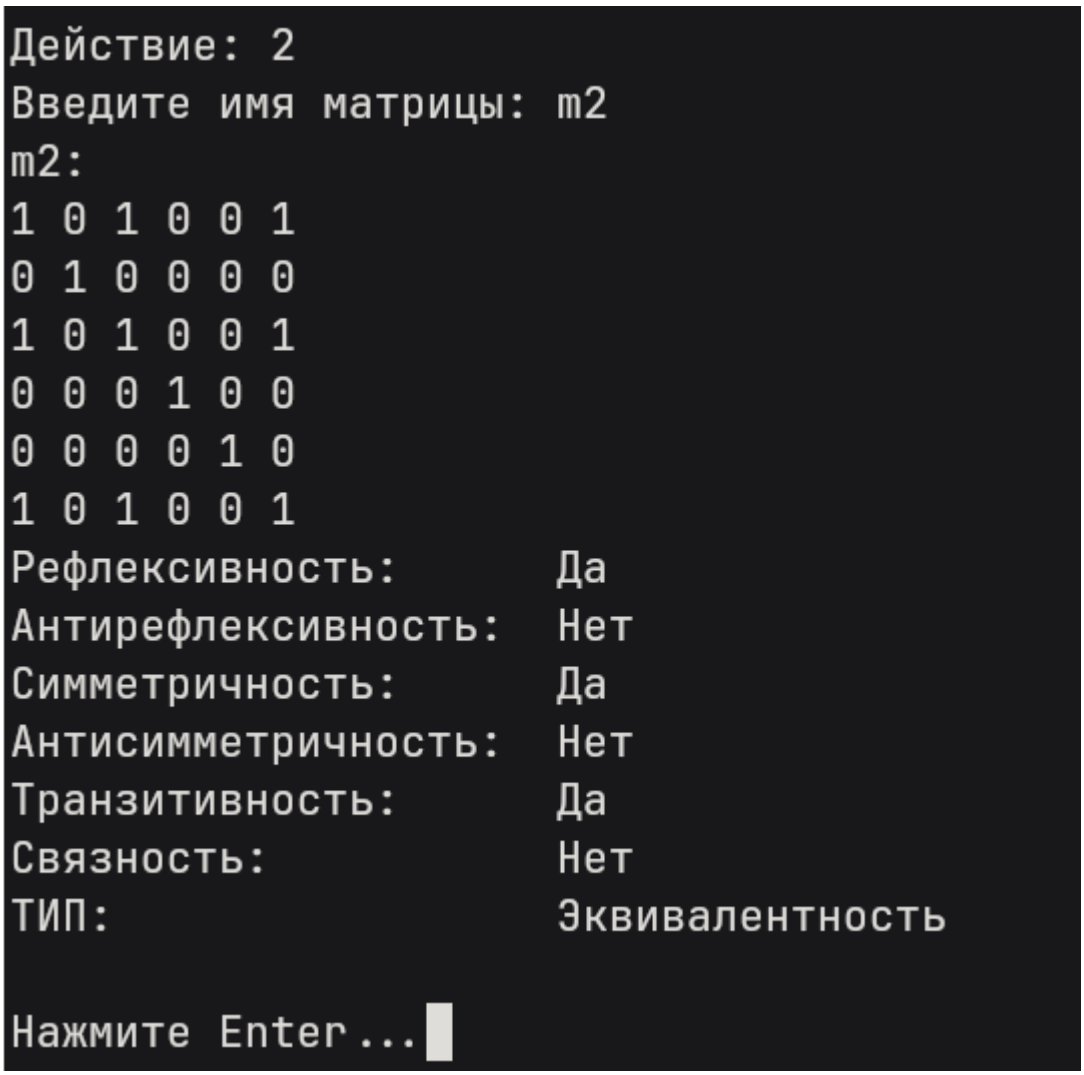


Рисунок 4 — Результат анализа матрицы m2 (Эквивалентность)

На рисунке 5 представлен результат анализа матрицы m3. Данное отношение является антисимметричным и транзитивным, однако свойство связности отсутствует, поэтому программа классифицировала его как «никакой».

```
Действие: 2
Введите имя матрицы: m3
m3:
0 0 1 0 0 1
1 0 0 0 0 0
0 1 0 0 0 1
1 1 1 0 0 0
1 1 1 1 0 0
0 1 0 1 1 0
Рефлексивность:      Нет
Антирефлексивность:  Да
Симметричность:      Нет
Антисимметричность:  Да
Транзитивность:      Нет
Связность:           Да
ТИП:                 Никакое из перечисленных

Нажмите Enter...
```

Рисунок 5 — Результат анализа матрицы m3 (никакой)

Для удобства проверки была реализована функция пакетного анализа. На рисунке 6 показан результат вывода свойств и типов сразу для всех загруженных в программу матриц (от m1 до m3).

```
Действие: 3
m1:
1 0 1 0 0 1
0 1 0 0 0 0
0 0 1 0 0 1
0 0 0 1 0 0
0 0 0 0 1 0
0 0 0 0 0 1
Рефлексивность:      Да
Антирефлексивность:  Нет
Симметричность:      Нет
Антисимметричность:  Да
Транзитивность:      Да
Связность:           Нет
ТИП:                 Частичный порядок

-----------------------------
m2:
1 0 1 0 0 1
0 1 0 0 0 0
1 0 1 0 0 1
0 0 0 1 0 0
0 0 0 0 1 0
1 0 1 0 0 1
Рефлексивность:      Да
Антирефлексивность:  Нет
Симметричность:      Да
Антисимметричность:  Нет
Транзитивность:      Да
Связность:           Нет
ТИП:                 Эквивалентность

-----------------------------
m3:
0 0 1 0 0 1
1 0 0 0 0 0
0 1 0 0 0 1
1 1 1 0 0 0
1 1 1 1 0 0
0 1 0 1 1 0
Рефлексивность:      Нет
Антирефлексивность:  Да
Симметричность:      Нет
Антисимметричность:  Да
Транзитивность:      Нет
Связность:           Да
ТИП:                 Никакое из перечисленных
```

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы было разработано программное обеспечение на языке Python для анализа свойств бинарных отношений, заданных в виде матриц смежности. Программа позволяет считывать исходные данные из текстовых файлов, корректно обрабатывать их и сохранять в памяти для дальнейшей работы.

В рамках работы был реализован полный набор проверок шести фундаментальных свойств отношений: рефлексивности, антирефлексивности, симметричности, антисимметричности, транзитивности и связности. На основе анализа комбинации этих свойств программа автоматически классифицирует отношения, определяя такие типы, как эквивалентность, строгий порядок, частичный или полный порядок.

Разработанный алгоритм не использует сторонние библиотеки для матричных вычислений, все проверки реализованы средствами самого языка. Для удобства пользователя создан интерактивный консольный интерфейс, позволяющий как загружать отдельные матрицы, так и проводить пакетный анализ всех доступных файлов с наглядным выводом результатов. Проведенное тестирование на предоставленных матрицах подтвердило корректность работы программы: она успешно справляется с поставленной задачей и верно определяет свойства и типы заданных отношений.

Приложение

Полный код лабораторной работы доступен по ссылке:

https://github.com/ZentoTAG/LABS/blob/main/LABS_DM/lab2/main.py

ПРИЛОЖЕНИЕ А

Полный код лабораторной работы доступен по ссылке:

https://github.com/ZentoTAG/LABS_DM/blob/main/lab_1/main.py